RLNC for Ethereum Block Propagation

Linearly-Homomorphic Signatures for Network Coding

[Author Name]

March 10, 2026

[Institution]

The Problem

Ethereum's Roadmap: Bigger Blocks

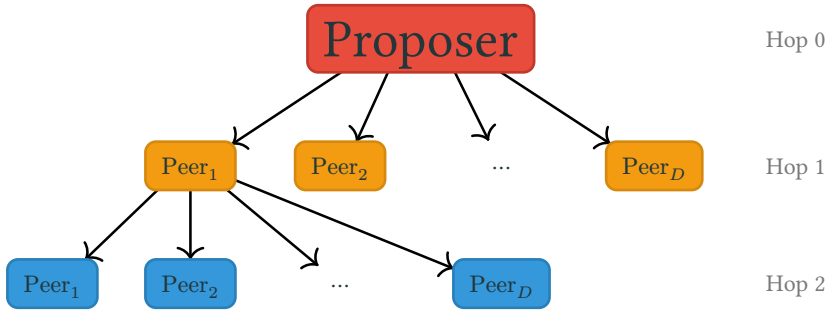
- Rollup-centric roadmap: L2s handle transactions, L1 provides **settlement + data availability**
- Each upgrade ships more blobs and raises gas limits
- Three compounding forces:
  1. **More blobs** (6 ⟶ 128+)
  2. **Higher gas limits** (60M ⟶ 200M+)
  3. **zkEVM** removes re-execution burden
- Core tension: the p2p layer was built for blocks an order of magnitude smaller

Era	Blobs	Data
Today (post-Fusaka)	14	2 MB
Glamsterdam	48	6.5 MB
Hegota	72	10 MB
zkEVM	128+	17+ MB

Gossipsub: How Blocks Travel Today

- Every node maintains a **mesh** of $D \approx 8$ peers
- **Full-block forwarding**: a node receives the *entire* block before forwarding to all mesh peers
- **Redundant delivery**: mesh connections overlap — nodes frequently receive duplicates
- Hops to reach all n nodes:

$$k = \left\lceil \frac{\ln n}{\ln D} \right\rceil \approx 5\text{–}6 \text{ hops}$$



The Scalability Wall

Propagation time over k hops:

$$T_{\text{gossipsub}} = k \cdot \left(L + \frac{B}{X} \right)$$

The Scalability Wall

Propagation time over k hops:

$$T_{\text{gossipsub}} = k \cdot \left(L + \frac{B}{X} \right)$$

Today ($B = 2$ MB):

$$T = 6 \times 170 = 1020 \text{ ms}$$

Hegota ($B = 10$ MB):

$$T = 6 \times 570 = 3420 \text{ ms}$$

zkEVM ($B = 17$ MB):

$$T = 6 \times 920 = 5520 \text{ ms}$$

The Scalability Wall

Propagation time over k hops:

$$T_{\text{gossipsub}} = k \cdot \left(L + \frac{B}{X} \right)$$

Today ($B = 2$ MB):

$$T = 6 \times 170 = 1020 \text{ ms}$$

Hegota ($B = 10$ MB):

$$T = 6 \times 570 = 3420 \text{ ms}$$

zkEVM ($B = 17$ MB):

$$T = 6 \times 920 = 5520 \text{ ms}$$

Maximum block size within a 4 s budget:

$$B_{\max} = \frac{T_{\text{budget}}}{k} \cdot X - L \cdot X \approx 11.7 \text{ MB theoretical} \rightarrow 5\text{–}6 \text{ MB practical}$$

Gossipsub cannot sustain the block sizes Ethereum's roadmap demands.

Random Linear Network Coding

Network Coding

Mixing Instead of Copying

Alice has a recipe: $(5, 3, 7)$ — grams of **Red**, **Green**, **Blue**.

Instead of sending pure pigments, she sends **mixtures** with labels:

Bucket	Label (ratios)	Contents	Total
To Bob	$(2, 1, 0)$	$2 \times 5 + 1 \times 3 + 0 \times 7$	13
To Carol	$(0, 1, 3)$	$0 \times 5 + 1 \times 3 + 3 \times 7$	24
To Dave	$(1, 0, 2)$	$1 \times 5 + 0 \times 3 + 2 \times 7$	19
To Eve	$(3, 2, 1)$	$3 \times 5 + 2 \times 3 + 1 \times 7$	28

Mixing Instead of Copying (cont.)

Dave's bucket is lost! Solve from Bob, Carol, Eve:

$$2\mathrm{R} + 1\mathrm{G} + 0\mathrm{B} = 13$$
$$0\mathrm{R} + 1\mathrm{G} + 3\mathrm{B} = 24$$
$$3\mathrm{R} + 2\mathrm{G} + 1\mathrm{B} = 28$$

Mixing Instead of Copying (cont.)

Dave's bucket is lost! Solve from Bob, Carol, Eve:

$$2\mathrm{R} + 1\mathrm{G} + 0\mathrm{B} = 13$$
$$0\mathrm{R} + 1\mathrm{G} + 3\mathrm{B} = 24$$
$$3\mathrm{R} + 2\mathrm{G} + 1\mathrm{B} = 28$$

- From Bob: $\mathrm{G} = 13 - 2\mathrm{R}$
- Substitute into Carol: $3\mathrm{B} = 11 + 2\mathrm{R}$
- Substitute into Eve $\longrightarrow$ $\mathrm{R} = 5$, $\mathrm{G} = 3$, $\mathrm{B} = 7$

Mixing Instead of Copying (cont.)

Dave's bucket is lost! Solve from Bob, Carol, Eve:

$$2\mathrm{R} + 1\mathrm{G} + 0\mathrm{B} = 13$$
$$0\mathrm{R} + 1\mathrm{G} + 3\mathrm{B} = 24$$
$$3\mathrm{R} + 2\mathrm{G} + 1\mathrm{B} = 28$$

- From Bob: $\mathrm{G} = 13 - 2\mathrm{R}$
- Substitute into Carol: $3\mathrm{B} = 11 + 2\mathrm{R}$
- Substitute into Eve $\longrightarrow$ $\mathrm{R} = 5$, $\mathrm{G} = 3$, $\mathrm{B} = 7$

Anyone can remix: Bob combines his + Carol's $\longrightarrow$ valid mixture $(2, 2, 3)$, value 37. No original pigments needed!

Paint	**RLNC**
Recipe $(5, 3, 7)$	Block $\longrightarrow$ N chunks
Pigment / Bucket	Chunk $\boldsymbol{v}_i$ / Coded chunk $\boldsymbol{w}$
Label (ratios)	Coefficient vector $\boldsymbol{b}$
Solve equations	Gaussian elimination over $\mathbb{F}_p$

A block is N vectors in $\mathbb{F}_p^M$: $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_N$

A **linear combination**:

$$\boldsymbol{w} = \sum_{i=1}^{N} b_i \cdot \boldsymbol{v}_i$$

Need N linearly independent coded chunks to decode.

Why random coefficients work:

$$\Pr[\text{dependent}] \leq \frac{1}{p} \approx 2^{-252}$$

With b_i sampled uniformly from $\mathbb{F}_p$, every random combination is independent with overwhelming probability.

RLNC Encoding

The proposer holds $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_N$.

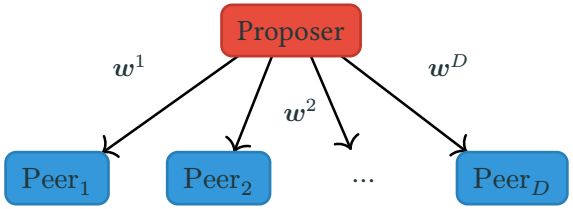
For each peer j:

1. Sample random $\boldsymbol{b}^{(j)} \leftsquigarrow \mathbb{F}_p^N$
2. Compute coded chunk:

$$\boldsymbol{w}^{(j)} = \sum_{i=1}^{N} b_i^{(j)} \cdot \boldsymbol{v}_i$$

3. Send $\left(\boldsymbol{w}^{(j)}, \boldsymbol{b}^{(j)}\right)$

Each peer gets a **different** random linear combination.



Re-encoding & Decoding

Re-encoding (at intermediate nodes):

Given L received coded chunks, sample $\alpha_1, \ldots, \alpha_L$:

$$\boldsymbol{w}' = \sum_{\ell=1}^{L} \alpha_\ell \cdot \boldsymbol{w}_\ell, \qquad b'_i = \sum_{\ell=1}^{L} \alpha_\ell \cdot b_{\ell,i}$$

Intermediate nodes operate **entirely on coded chunks** — no original data needed.

Re-encoding & Decoding

Re-encoding (at intermediate nodes):

Given L received coded chunks, sample $\alpha_1, \ldots, \alpha_L$:

$$\boldsymbol{w}' = \sum_{\ell=1}^{L} \alpha_\ell \cdot \boldsymbol{w}_\ell, \qquad b'_i = \sum_{\ell=1}^{L} \alpha_\ell \cdot b_{\ell,i}$$

Intermediate nodes operate **entirely on coded chunks** — no original data needed.

Decoding (once N independent chunks collected):

$$\begin{pmatrix} \boldsymbol{w}_1 \\ \vdots \\ \boldsymbol{w}_N \end{pmatrix} = \boldsymbol{B} \cdot \begin{pmatrix} \boldsymbol{v}_1 \\ \vdots \\ \boldsymbol{v}_N \end{pmatrix} \qquad \rightarrow \qquad \begin{pmatrix} \boldsymbol{v}_1 \\ \vdots \\ \boldsymbol{v}_N \end{pmatrix} = \boldsymbol{B}^{-1} \cdot \begin{pmatrix} \boldsymbol{w}_1 \\ \vdots \\ \boldsymbol{w}_N \end{pmatrix}$$

The Integrity Problem

Pollution attack: a malicious node injects a garbage coded chunk.

- The garbage is linearly combined with legitimate chunks at every downstream node
- When a node collects N chunks and inverts the matrix, it recovers **the wrong block**
- The corruption is **invisible** until decoding — too late

Verification is hard: receivers only see coded chunks, never the originals.

How does a receiver know a coded chunk is legitimate?

Pedersen Commitments

Let $G_1, G_2, ..., G_M \in \mathbb{G}_1$ be public generators on BLS12-381.

Construction — commit to vector $\boldsymbol{v} = (a_1, ..., a_M)$:

$$C(\boldsymbol{v}) = \sum_{j=1}^{M} a_j \cdot G_j \quad \in \mathbb{G}_1$$

A single $\mathbb{G}_1$ point — 48 bytes compressed.

Pedersen Commitments

Let $G_1, G_2, ..., G_M \in \mathbb{G}_1$ be public generators on BLS12-381.

Construction — commit to vector $\boldsymbol{v} = (a_1, ..., a_M)$:

$$C(\boldsymbol{v}) = \sum_{j=1}^{M} a_j \cdot G_j \quad \in \mathbb{G}_1$$

A single $\mathbb{G}_1$ point — 48 bytes compressed.

Homomorphic property:

$$C(\alpha \boldsymbol{v} + \beta \boldsymbol{u}) = \alpha \cdot C(\boldsymbol{v}) + \beta \cdot C(\boldsymbol{u})$$

Linearity of scalar multiplication gives us this for free. A commitment to a coded chunk equals the same linear combination of the original commitments.

Pedersen for RLNC: Verification

Proposer computes $C_i = C(\boldsymbol{v}_i)$ for each chunk and signs $(C_1, \ldots, C_N)$ with BLS $\longrightarrow \sigma$.

Each packet carries: $(\boldsymbol{w}, \boldsymbol{b}, \{C_1 \ldots C_N\}, \sigma)$

3-step verification pipeline:

1. **BLS signature check** — verify σ on $(C_1, \ldots, C_N)$ with proposer's pk
2. **Commitment check** — verify $C(\boldsymbol{w}) \stackrel{?}{=} \sum_{i=1}^{N} b_i \cdot C_i$
3. **Independence check** — is $\boldsymbol{b}$ linearly independent of previously received vectors?

If any check fails $\longrightarrow$ discard. Pollution attacks detected and rejected.

The Commitment Overhead Problem

Component	Size	Per packet?
$\boldsymbol{w}$ (coded data)	$32M$ bytes	Unique
$\boldsymbol{b}$ (coefficients)	$32N$ bytes	Unique
$C_1 \ldots C_N$ (commitments)	$48N$ bytes	**Identical**
σ (BLS signature)	96 bytes	**Identical**

N commitments repeated in **every** packet at **every** hop.

$N = 10$: **128 MB** redundant traffic $N = 64$: **819 MB** redundant traffic

What if we could replace N commitments with a single signature?

Linearly-Homomorphic Signatures

Bilinear Pairings

A **bilinear pairing** maps pairs of curve points to a target group:

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$$

Bilinearity: $e(a \cdot P, b \cdot Q) = e(P, Q)^{ab}$

Group	Size (BLS12-381)
$\mathbb{G}_1$	48 bytes
$\mathbb{G}_2$	96 bytes
$\mathbb{G}_T$	576 bytes

BLS signature (motivating example):

- $\sigma = sk \cdot H(m) \in \mathbb{G}_2$
- Verify: $e(G_1, \sigma) \stackrel{?}{=} e(pk, H(m))$
- Works because bilinearity moves sk across arguments

Key insight: pairings let you check scalar relationships between secret keys and signed data **without revealing the keys**.

LHSS Intuition: Certified Paint

Callback to Alice's paint analogy:

- The proposer **certifies** each base pigment with a cryptographic stamp (σ_i)
- Anyone can blend certified pigments and **derive** a valid stamp for the blend — without the proposer's stamp pad (secret key)
- But you **cannot** certify a color not mixed from the originals (security)

Three formal properties:

1. **Verifiable** with public key only
2. **Composable** — combine σ_1, σ_2 into σ' without sk
3. **Controlled** — only within the linear span of signed vectors

Combine signatures WITHOUT the secret key.

LHSS: The Four Algorithms

$\text{Setup}(1^\lambda, M, N)$
- $\longrightarrow (sk, pk)$

$\text{Sign}(sk, id, \boldsymbol{m}, i)$
- $\longrightarrow$ signature σ

$\text{Combine}(pk, id, \{(a_i, \sigma_i)\})$
- $\longrightarrow$ combined σ (public key only!)

$\text{Verify}(pk, id, \boldsymbol{v}, \sigma, \boldsymbol{a})$
- $\longrightarrow$ accept / reject

Correctness:

1. Direct signatures verify: Sign $\rightarrow$ Verify $= 1$
2. Combined signatures verify:

$$\text{Verify}\left(pk, id, \sum_i a_i \boldsymbol{m}_i, \text{Combine}(\ldots), \sum_i a_i \boldsymbol{a}_i\right) = 1$$

BFKW Construction: Setup & Sign

Basis vector trick: augment each chunk before signing

$$\boldsymbol{m}'_i = (\boldsymbol{v}_i \parallel \boldsymbol{e}_i) \in \mathbb{F}_p^{M+N}$$

Linear combination yields: $\sum b_i \cdot \boldsymbol{m}'_i = (\boldsymbol{w} \parallel \boldsymbol{b})$ — data **and** coefficients bound together.

BFKW Construction: Setup & Sign

Basis vector trick: augment each chunk before signing

$$\boldsymbol{m}'_i = (\boldsymbol{v}_i \parallel \boldsymbol{e}_i) \in \mathbb{F}_p^{M+N}$$

Linear combination yields: $\sum b_i \cdot \boldsymbol{m}'_i = (\boldsymbol{w} \parallel \boldsymbol{b})$ — data **and** coefficients bound together.

Setup:

- $sk \leftsquigarrow \mathbb{F}_p, \quad pk = sk \cdot G_1 \in \mathbb{G}_1$

Sign chunk $\boldsymbol{v}_i$ with index i:

1. Compute hash points: $H_s = H(id, s) \in \mathbb{G}_2$ for $s = 1, \ldots, M+N$
2. $$P = \sum_{s=1}^{M+N} m'_s \cdot H_s = \mathrm{MSM}(H, [\boldsymbol{v}_i \parallel \boldsymbol{e}_i]) \in \mathbb{G}_2$$
3. $$\sigma = sk \cdot P \in \mathbb{G}_2$$

BFKW Construction: Combine & Verify

Combine — MSM in $\mathbb{G}_2$:

$$\sigma' = \sum_{i=1}^{k} a_i \cdot \sigma_i \in \mathbb{G}_2$$

Correctness: $\sigma' = \sum a_i \cdot sk \cdot \mathrm{MSM}(H, \boldsymbol{m}'_i) = sk \cdot \mathrm{MSM}(H, \sum a_i \cdot \boldsymbol{m}'_i) = sk \cdot \mathrm{MSM}(H, (\boldsymbol{w} \parallel \boldsymbol{b}))$

BFKW Construction: Combine & Verify

Combine — MSM in $\mathbb{G}_2$:

$$\sigma' = \sum_{i=1}^{k} a_i \cdot \sigma_i \in \mathbb{G}_2$$

Correctness: $\sigma' = \sum a_i \cdot sk \cdot \mathrm{MSM}(H, \boldsymbol{m}'_i) = sk \cdot \mathrm{MSM}(H, \sum a_i \cdot \boldsymbol{m}'_i) = sk \cdot \mathrm{MSM}(H, (\boldsymbol{w} \parallel \boldsymbol{b}))$

Verify: compute $P = \mathrm{MSM}(H, [\boldsymbol{w} \parallel \boldsymbol{b}])$ and check:

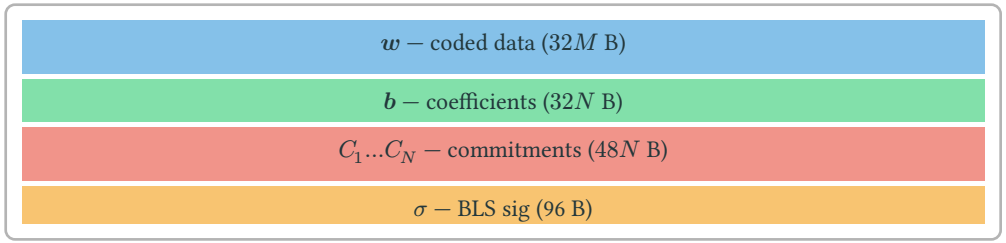
$$e(G_1, \sigma) \stackrel{?}{=} e(pk, P)$$

Bilinearity moves sk from $\mathbb{G}_2$ to $\mathbb{G}_1$:

$$e(G_1, sk \cdot P) = e(sk \cdot G_1, P) = e(pk, P) \quad \checkmark$$

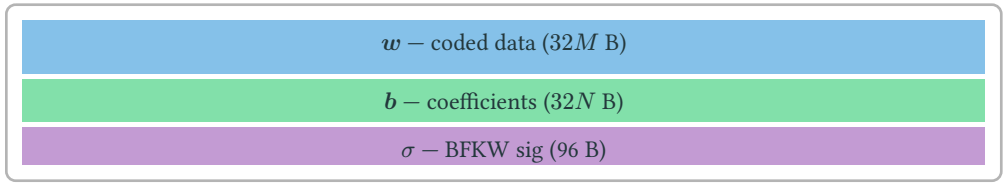
Pedersen vs BFKW: Final Comparison

Pedersen packet:



Overhead: $80N + 96$ bytes

BFKW packet:



Overhead: $32N + 96$ bytes

Metric	Pedersen	BFKW	Winner
Comm / packet	$80N + 96$ B	$32N + 96$ B	**BFKW**
Network overhead	$\mathcal{O}(nDN)$	$\mathcal{O}(nD)$	**BFKW**
Proposer compute	N M-MSMs in $\mathbb{G}_1$	N M-MSMs + D N-MSMs in $\mathbb{G}_2$	Pedersen
Receiver compute	2 MSMs	1 MSM + 1 multi-pairing	Comparable
Setup	Trusted generators	Key pair only	**BFKW**

LHSS wins on communication at the cost of slightly more computation.

Thank You